

Labscript	2
Labscript Overview	3
Why Labscript?	4
Understanding the Labscript Environment	7
Labscript-suite Package Structure	8
GitHub Sync of labscript-suite Folder	11
labscript-suite Folder	14
Sequences Files	17
labscript-devices (Backend) Folder	19
How-To Guides	21
Installation Guide	22
Startup Procedure	26
How to Write the Connection Table	28
Convention on connection_table.py	33
Connecting hardwares to Labscript	34
Connecting devices using PyVISA, PySerial, and Pylablib	35
External Software Devices	37
NuVu camera notes	42
Communicating with LabView	43
How to set measurement parameters	44
Template	47
Package dependence issue for installing labscript	51
Debugging Issues	53
Intro to NI DAQ	54

Labsript

Adding Notes on Confluece

- For this free account, you can't copy-paste figures/files. Save on your computer or screenshot, then either drag or use the insert function
- When adding a debugging note, copy and edit the [template](#)
- See this page [▲ Insert the code block macro | Confluence Cloud | Atlassian Support](#) for how to insert code blocks
- “Related content” shown on bottom of the pages are automatically generated bullshit - ignore them

Content Search

Q Search our help articles

Labscript Overview

- a chart outlining the high level usage of the `labscript-suite` and flow between each of the applications

python + `runmanager` globals → `runmanager` compiler → `runviewer`

verification → initial compiled .h5 file with hardware instructions → `blacs` → final .h5 file

with saved data → `lyse` → view processed analysis data

Why Labscript?

Pseudoclock

There are several electronics boards that work as a pseudoclock. PineBlaster is deprecated (the board it was using is no longer supported by the manufacturer), so replaced with PrawnBlaster which turns a Raspberry Pi Pico into a time counter. PulseBlaster works like the same, but it works with Spincore Pulseblaster DDS-II-300-AWG.

A Pico board can be max. four independent pseudoclocks

→ We use Raspberry Pi Pico with [PrawnBlaster](#).

Details of pseudoclock can be found in a [PhD thesis](#) of a Labscript developer. Their main purpose is to “correctly capture precision timing of laser switching in **optical pumping** (~10us timescales) with minimum data load for operation.”

Good quote from the thesis:

In order to maintain precise timing of updates to hardware outputs, I/O devices typically prefill an onboard buffer with [hardware instructions](#) and step through them according to a provided clock tick. The most common approach is then to step through these instructions (after receiving an appropriate trigger to start) with a clock that ticks at a constant rate (see figure 2.1), since such clocks are easy to generate and can be trivially included in the design of the hardware. However, this becomes unwieldy for experiments where the ratio between the experiment length and the required timing resolution is large. For ultra-cold atom experiments, this ratio is at least 10^6 (10 microsecond timing resolution during experiments of 10 seconds) and may even hit 10^9 (100 nanosecond timing resolution during experiments of 100 seconds). Each output channel driven by a constant rate clock would then require somewhere between 10^6 and 10^9 hardware instructions in order to achieve the desired timing resolution and duration. This could be as much as 2 gigabytes of data per

channel (for a 16-bit analog output). However, most output devices do not have the onboard memory to support such large instruction sets. While some manufacturers instead stream the hardware instruction set to the device during the experiment execution, thus avoiding onboard memory limitations, this scheme is difficult to implement well (and so only large manufacturers such as [National Instruments \(NI\)](#) attempt it). It comes with a risk of failing to stream data fast enough if resources or bus bandwidth on the PC are consumed by other processes, and the number of hardware instructions is still limited by the memory of the control PC.

We use the concept of a pseudoclock to work around these memory limitations, by providing a mechanism to vary the time between instructions. This allows us to maintain high timing resolution only during the segments of the experiment where it is necessary, reducing the required number of instructions to within the capabilities of most devices with small onboard memory buffers.

Labscript API in Chapter 5:

through on the edges of a digital clocking signal from a [pseudoclock](#). Hardware instructions are typically esoteric, thus difficult for humans to both create them, as well as read them, by hand. The aim of the labscript [API](#) is to provide a high level interface for lab users, which will in turn create the low level instructions needed for each piece of hardware, including the clocking signal.

To summarize,

- The pseudoclock is essentially a lossless-ly compressed clock signal that is used to step through the programmed instructions on your devices. Extensive work has gone into developing and benchmarking the timing. [🤖 Experimental timing and control using microcontrollers](#)
- If you are running into problems where your shot length is limited due to the hardware device being programmed not being able to support the number of instructions, a pseudoclock may be of interest to you

BLACS

- Interfaces between 2 modes of operation: the execution of shots under hardware timing, and the manual control of hardware devices (by users) via the GUI. The GUI is split into 2 sections:
 - a. The queue of shots to execute
 - b. The main GUI for each device allowing you to manually control the output of the hardware devices when not running shots.
 - c. We can control each of the signals independently, lock values that are sensitive and shouldn't be changing

Runmanager

Runmanager is the primary means of defining and managing the set of experiment parameters (global variables) used in the labscript experiment logic. Runmanager also handles the creation of each hdf5 shot file, and the invocation of labscript via the execution of a user specified labscript experiment logic file. BLACS is designed to execute shot files on an apparatus. After shots are generated by runmanager, they can be sent to BLACS where they are queued and executed in sequence by BLACS. BLACS coordinates the programming of hardware devices, including via secondary control systems, and the saving of results to the shot file. Once a shot has been executed on the apparatus, the shot file is forwarded to lyse for analysis. Lyse runs a series of user-defined analysis routines (also written in Python) that access one or more shot files and perform the requested analysis.

Data storage

HDF5 (.h5) file is very useful when you want to store huge data with small load on the desktop memory (but it required a couple of steps to open and read on a screen...you can use)

A .h5 file is created in runmanager when you start the sequence. The file receives all the information as the experiment proceeds. Always refer to [Labscript page](#) for details.

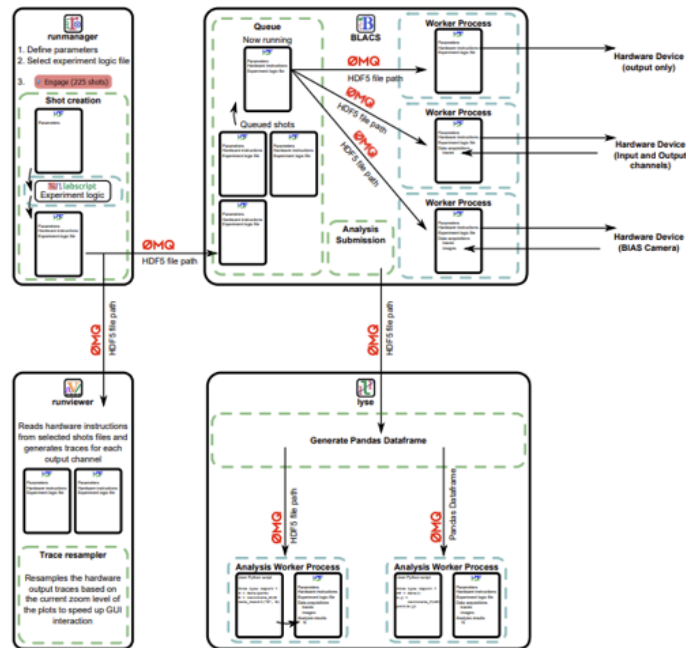


Figure 4.1: The flow of data between components of the labscript suite is shown. Green dashed lines denote threads within a program while blue dashed lines denote a worker process. Shots are born in runmanager, where the HDF5 shot file is created containing the set of experiment parameters. Runmanager then executes the experiment logic script in a worker process, which uses the labscript application programming interface (API) to generate hardware instructions. Shots are passed to BLACS and/or runviewer. BLACS executes shot files from a queue and any acquired data is saved in the shot file. After execution, shot files are then passed to lyse, where they are analysed by a series of user specified analysis routines (Python scripts), which may save analysis results in the shot file. References to a shot file are passed via ZeroMQ, both between components of the labscript suite and between a component and its worker processes.

Log files can be found in /c/Users/...../labscript-suite/logs

1. Contains log files for each of the applications
2. **BLACS.log** is particularly useful for understanding where BLACS ran into an error or got stuck

Understanding the Labsript Environment

This set of wiki pages is designed to provide you with a comprehensive understanding of the `labscript-suite` s structure, organization, and key components.

As you are reading the documentation, you may find it helpful to refer to the actual `labscript-suite` install on the Main Experiment Control computer. At the very least, follow along with the `RaX-labscript/userlib` folder found here: [GitHub - shafin.](#)

What You'll Find in This Section

1. `labscript-suite` Folder Structure and GitHub Synchronization
 - The basic and developer installation folder structures
 - Understand how local folders sync with GitHub repositories
 - Discover which folders remain local and which are shared across installations
2. `labscript-suite` Folder Descriptions
 - Detailed explanations of core folders like `labconfig` and `userlib`
 - Information on supporting folders and developer installation components
 - Insights into PC-specific folders and their roles
3. **Organizing** `labscriptlib`
 - Best practices for structuring your experiment control scripts
 - Overview of the current folder organization for sequences, subsequences, and globals
 - Tips for maintaining an efficient and scalable experiment codebase

Labscript-suite Package Structure

There are 4 github repos in the [labscript-suite package](#):

- labscript-suite ← This is a folder name in the labscript-suite package, which is very confusing
- blacs
- labscript-utils
- labscript-devices

The latter 3 are backend.

The first (labscript-suite) is user facing and tracks userlib. labscript-suite on two computers, “computer A” and “computer B”, point to the same remote repo (RaX-labscript on github  [GitHub - shafinulh/RaX-labscript: The labscript-suite folder for the RaX experiment](#)). the key distinction is that on computer A you will be modifying labscript-suite/userlib/labscriptlib/computer A part of the repo and computer B will be modifying labscript-suite/userlib/labscriptlib/computer B part of the repo.

GitHub repos:

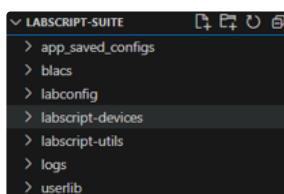
(1) labscript-suite repo

- userlib
- gitignore

(2) BLACS

(3) labscript-devices

(4) labscript-utils



Important Folders

1. Labconfig

- a. Contains the "COMPUTER_NAME.ini" file

```
labconfig > shafinulhini
1  [DEFAULT]
2  apparatus_name = example_apparatus
3  shared_drive = C:
4  experiment_shot_storage = %(shared_drive)s\Experiments\%(apparatus_name)s
5  userlib = %(labscript_suite)s\userlib
6  pythonlib = %(userlib)s\pythonlib
7  labscriplib = %(userlib)s\labscriplib\%(apparatus_name)s
8  analysislib = %(userlib)s\analysislib\%(apparatus_name)s
9  app_saved_configs = %(labscript_suite)s\app_saved_configs\%(apparatus_name)s
10 user_devices = user_devices
```

1. *Apparatus_name* should be set to the name describing the experiment
2. *Shared_drive* will be the path where shots should be saved
3. **Userlib**

```
> analysislib
> labscriplib
> pythonlib
> user_devices
```

1. **Analysislib/%(apparatus_name)** is where all the analysis scripts to be used in lyse are stored.
2. **Labscriplib/%(apparatus_name)** is where all the experiment control files are stored.

```
✓ labscriplib
> __pycache__
> common
✓ example_apparatus
> __pycache__
◆ connection_table_IMAQdx_re... 7
◆ connection_table.py
◆ example_experiment.py
◆ example_IMAQdx_remote.py
◆ __init__.py
```

- i. **Connection_table.py** is used to define and represent all the hardware connections in the experiment
- ii. **%(experiment_name).py** is defines the experiment logic that is later compiled into hardware instructions by runmanagercd ..

Other Folders

1. **app_saved_configs**

- a. The default folder that contains automatically created files to store the configurations of each app (runmanager, BLACS, runviewer)

2. **Blacs**

- a. The backend code of the BLACS application (runtime engine)

3. **Labscript-devices**

- a. The backend code for the device drivers.
- b. This is where we define device interactions for each of the manual or buffered mode tasks.

4. **Labscript-utils**

- a. The backend code that contains utility functions.

- b. Probably won't need to be touched much

5. Logs

- a. Log files for each of the applications

- b. BLACS.log is particularly useful for understanding where BLACS ran into an error or got stuck

GitHub Sync of labscript-suite Folder

- [Developer Installation](#)
- [Main labscript-suite folders](#)
 - [Remote Synchronization](#)
 - [Local-Only Folders](#)
 - [Custom Local Folders](#)
 - [Synchronized Content](#)
- [Developer Install Folders](#)
 - [Key Points:](#)

Developer Installation

After the regular install, you may need to perform a developer installation if you want modifications to the default labscript backend flow (which is what Shafin worked on to improve BLACS speed). The developer installed folders are:

- `blacs`
- `labscript-devices`
- `labscript-utils`

Main `labscript-suite` folders

Remote Synchronization

Each computer with **Labscript** installed has its `labscript-suite` folder linked to a remote origin on GitHub: [GitHub - shafinulh/RaX-labscript: The labscript-suite folder for the RaX experiment](#).

Local-Only Folders

The `.gitignore` file in the repository specifies that certain folders should not be synchronized with the RaX remote repository linked above:

- `app_saved_configs/`
- `labconfig/`
- `logs/`
- `blacs/`
- `labscript-devices/`
- `labscript-utils/`

These folders remain local to each computer and are not pushed to or pulled from the remote.

 The last 3 developer install folders are also remote folders, but they are NOT synched with the `RaX-labscript` repo. Keep reading to understand how those are synchronized to their own separate repos.

Custom Local Folders

When creating a new local folder (e.g., `network_testing` for computer-specific scripts):

1. Create the folder within the `labscript-suite` directory.
2. Add the folder name to the `.gitignore` file to keep it local.

If you later decide to make a local folder accessible across all computers, you can remove it from `.gitignore` and push it to the remote repository.

Synchronized Content

The primary synchronized content is the `userlib` folder. This folder contains subfolders for each `apparatus_name`. Keeping all RaX `userlibs` in one repository allows for easy reference and file copying between different computers. The computers cannot all share one common repository since they will be controlling different hardware and corresponding connection tables.

 **Note:** The remote synchronization of the `userlib/labscript-suite` is primarily for development and testing purposes. In the final environment, you typically won't have multiple computers running different hardware simultaneously like we currently do with **Lyman29** and **Main Exp Control**.

Developer Install Folders

The developer install folders (`BLACS`, `labscript-devices`, and `labscript-utils`) contain the main backend code. These folders are linked to separate remote repositories:

-  [GitHub - shafinulh/blacs](#): **blacs** supervises the execution of experiments controlled by the *labscript suite*. It manages experiment queuing and hardware-timed execution, and provides manual control over devices between shots.
-  [GitHub - shafinulh/labscript-devices](#): A modular and extensible plugin architecture to control experiment hardware using the *labscript suite*.

- [GitHub - shafinulh/labscript-utils](#): Shared modules used by the *labscript suite*. Includes a graphical exception handler, debug tools, configuration management, cross platform filepaths conversions, unit conversions and custom GUI widgets.

Key Points:

1. These repositories define the core functionality of Labscript and are not specific to individual computer installations like the `labscript-suite` previously discussed. They essentially determine the version of Labscript that runs "under the hood" on any given machine.
2. Each backend repository has multiple branches:
 - `default-labscript` : the original backend
 - `master` : Stable, optimized backend
 - Feature branches: For new experimental feature development
3. These are shared remote folders - when making local changes:
 - Push changes to the remote repository
 - Other computers can pull from remote to see those changes.

labscript-suite Folder

This document outlines the key folders and files within the `labscript-suite` directory. This directory should generally not be moved after `labscript` has been installed.

- Core Folders
 - 1. Labconfig
 - Important Configuration Parameters:
 - Note on Path Modifications:
 - 2. Userlib
 - 2.1 analysislib/%(apparatus_name)
 - 2.2 labscriptlib/%(apparatus_name)
- Supporting Folders
 - 3. app_saved_configs
 - 4. Logs
- Developer Installation Folders
 - 5. BLACS
 - 6. Labscript-devices
 - 7. Labscript-utils
- PC-Specific Folders

Core Folders

1. Labconfig

Contains configuration files for the application, including:

`{COMPUTER_NAME}.ini`

This is the main configuration file. Key sections include:

```

1 [DEFAULT]
2 apparatus_name = main_experiment_control
3 shared_drive = C:
4 experiment_shot_storage = %(shared_drive)s\Experiments\%(apparatus_name)s
5 userlib = %(labscript_suite)s\userlib
6 pythonlib = %(userlib)s\pythonlib
7 labscriptlib = %(userlib)s\labscriptlib\%(apparatus_name)s
8 analysislib = %(userlib)s\analysislib\%(apparatus_name)s
9 app_saved_configs = %(labscript_suite)s\app_saved_configs\%(apparatus_name)s
10 user_devices = user_devices
11
12 [paths]
13 connection_table_h5 = %(experiment_shot_storage)s\connection_table.h5
14 connection_table_py = %(labscriptlib)s\connection_table.py

```

Important Configuration Parameters:

- `apparatus_name` : Set this to a name that describes your specific experiment.
- `shared_drive` : Set this to the path where experiment shots will be saved.

Note on Path Modifications:

All other paths can be modified as needed. If you choose to modify file names (i.e.

`connection_table` file name or `user_devices` folder), remember to update the corresponding entries in this configuration file.

2. Userlib

The primary location for user-created scripts and analysis tools.

2.1 analysislib/%(apparatus_name)

Stores all analysis scripts to be used in the Lyse application.

2.2 labscriplib/%(apparatus_name)

Contains all experiment control files, including:

- `connection_table.py` : Defines and represents all hardware connections in the experiment
- **experiment sequence scripts** `{*.py}` : Defines the experiment logic that is later compiled into hardware instructions by `runmanager`
- **experiment sequence globals** `{*.h5}` : Defines the global variables that are used during the `runmanager` compilation

Supporting Folders

3. app_saved_configs

Default folder for automatically created files storing configurations of each application (Runmanager, BLACS, Runviewer).

4. Logs

Contains log files for each application. Notable files include:

- `BLACS.log` : Particularly useful for troubleshooting BLACS errors or issues

Developer Installation Folders

The application folder also includes developer installations of some backend components to customize Labscript flow:

5. BLACS

Backend code for the BLACS application (runtime engine). This is where the state machine of the buffered mode execution is defined.

6. Labscript-devices

Backend code for device drivers. This is where device interactions for manual or buffered mode tasks are defined.

7. Labscript-utils

Backend code containing utility functions and features used throughout BLACS and labscript-devices. Generally doesn't require frequent modifications.

These folders contain the core functionality of the application and are typically not modified by end-users.

PC-Specific Folders

Some folders may be specific to individual PC setups. These are created during the installation/configuration process of the `labscript-suite` and will be unique to each computer although the structure will be the same.

Sequences Files

This explains the current folder structure for (apparatus_name) folder in userlib. This is what I suggest using, but is by no means the only way to go about it. Real example can be found in

[RaX-labscript/userlib/labscriptlib/main_experiment_control at master · shafinulh/RaX-labscript](#).

labscript Folder Structure

Our experiment control application uses the following folder structure:

```
1 experiment_specific_folder_name/
2 |— sequences/
3 |   |— device_testing/
4 |— sequences_globals/
5 |— subsequences/
6 |— subsequences_globals/
7 |— connection_table.py
```

Detailed Description

1. sequences/

- Scripts that define the high-level logic of the experiment shot. These scripts are built up using labscript primitive operations for each device, along with more complex custom defined subsequences. --In short, it has scripts that define the execution sequence of experiments.

1.1 device_testing/

- Scripts demonstrating individual functionality of different lab devices, to isolate and test specific hardware components.

2. sequences_globals/

- .h5 files with global variables used in the sequences.
- I have been making a globals file for each of the sequences.

3. subsequences/

- A single script `subsequences.py` that defines subsequence and utility functions serving as building blocks for larger experiment sequences. --In short, it has utility functions for building up experiment sequences.
- These functions are imported as reusable components in multiple sequences

4. subsequences_globals/

- .h5 files with global variables specific to subsequences.
- I have been making globals files for each of the subsequences defined in `subsequences.py`. The name of the globals file matches that of the subsequence

function name.

5. **connection_table.py**

- Defines all hardware connections and configurations for the lab setup.

NOTE: the ENTIRE `labscriptlib/%(apparatus_name)` is saved in the h5 after each shot. Therefore, spreading an experiment sequence across multiple files (subsequences.py, utils.py, etc.) is not a problem.

Writing/Editing a Sequence File

1. Start by examining and potentially modifying the `connection_table.py` file to ensure it accurately represents your hardware setup.
2. Create new sequence scripts in the `sequences/` folder or modify existing ones to define your experiment procedures.
3. Use the `subsequences/subsequences.py` file to create reusable functions that can be incorporated into your main sequences.
4. Store any global variables needed for your sequences in the appropriate `*_globals/` folders.
5. When testing individual devices, create or use scripts in the `sequences/device_testing/` folder.

Best Practices

- Add comments to explain complex operations.
- When creating new global variables, consider their scope and place them in the appropriate `*_globals/` folder.
- Regularly backup your scripts and maintain version control.

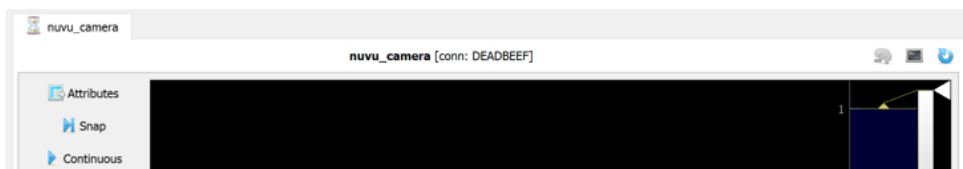
labscript-devices (Backend) Folder

Being familiar with the files that make up a `labscript-device` can be useful for debugging, understanding where to add/modify functionality, and even adding your own custom devices.

This explains the commonly used folders and files in `labscript-device`, how they work and why they are there. (Details can be found here: [📖 Devices — labscript suite | labscr ipt-devices](#))

Each device is made up of 3 classes that handle the various tasks.

- `labscript_device` (derives from `IntermediateDevice/Device`)
 - Defines the interface for the API for the device
 - configuring the device settings and properties
 - the high level commands (e.g. `A0.ramp`, `D0.go_high`, `camera.expose`)
 - Defines how the high level commands are translated to hardware instructions that are saved in the h5 file
- `BLACS_tab` (derives from `DeviceTab`)
 - Defines the graphical tab that is present in the **BLACS** GUI. This tab provides graphical widgets for controlling hardware outputs and visualizing hardware inputs.
 - configures which functions are run when each button is pressed
- `BLACS_worker` (derives from `Worker`)
 - Defines the software control to the hardware. Any action that involves the hardware will invoke the corresponding function inside the `BLACS_worker` file to make the driver call to the hardware. Perhaps this is best exemplified using some examples:



- ▪ You interface with the camera GUI and click the `snap` button. This schedules the `snap` worker class function that configures the camera (tells the camera to acquire an image), retrieves the images, and then stops the acquisition.

- If you instead press the continuous button, that will call a different function in the worker class that makes a driver call to tell camera to continuously acquire and starts up a thread to constantly retrieve those images. upon pausing the continuous, we will invoke the worker class function that make the driver call to stop acquiring and suspends the image retrieval thread
- you write a shot to use the NI card, you put the shot in the queue to execute it. this implicitly calls the worker class function to transition device buffered. this function unpacks the h5 file to know what hardware instructions to execute, and then configures a task using the NI DAQmx driver to load the NI card buffer with those instructions.

How-To Guides

Installation Guide

Configuring the Python Environment:

We will use an environment manager to organize the files, packages, and package dependencies of our labscript installation. We are currently using miniconda since we will be installing most of the packages ourselves, but a full Anaconda distribution could also be used just fine.

Install miniconda

- [Miniconda - Anaconda](#) (This can be Anaconda instead. Anaconda is usually better if you have a large capacity on your desktop and you are not so familiar with computer science.)
- Command line install:

```
1 curl https://repo.anaconda.com/miniconda/Miniconda3-latest-Windows-x86_64.exe -o
  miniconda.exe
2 start /wait "" miniconda.exe /S
3 del miniconda.exe
```

Making conda visible in your terminal

```
1 conda init
```

Installing Labscript

1. Define or grab an `environment.yml` file

```
% environment.yml
```

```
1 name: labscript
2 channels:
3   - defaults
4   - conda-forge
5   - labscript-suite
6 dependencies:
7   - python==3.11.9
8   - labscript-suite
9   - matplotlib
10  - numpy=1.26.4
11  - pyzmq=23.2.0
12  - pyqt
13  - pip
```

- It is important to create the environment from a `.yml` file that defines all our packages such that all the package dependencies can be resolved automatically and installed all at once.

- incrementally adding packages to the environment may cause issues with package versions and dependencies.

(Note that we had a package version problem when a naive pip installation of labscript-suite was attempted in the conda environment - see [this page](#). That is why we created the environment.yml file. Details on the labscript-suite install and setting up the 4 applications: 

[Developer installation \(Python Package Index\) — labscript suite | the labscript suite](#) .)

Note 2025/02/21 (AJ+SF): Numpy>=2.0 runs into a bug with the Nuvu camera code in `NuvuCamera\blacs_workers.py` , specifically because the function `np.string_()` has been deprecated (we should use `np.bytes_()` if running Numpy>=2.0). For now to avoid modifying the blacs Nuvu files, we just explicitly require `numpy=1.26.4`.

2. Make the environment

```
1 conda env create -f ./environment.yml
```

- This will create a a conda environment with the name listed under the `.yml` file.

(Update the environment)

1. Modify your environment.yml file to reflect the change you want to make in the environment (e.g. changing python version) (The environment may need to be activated to run this?)

```
2. 1 conda env update -f ./environment.yml --prune
```

Activate the environment

```
1 conda activate labscript
```

First time Labscript setup

- creating the above environment will result in labscript to be properly installed
- activate the labscript environment
- automatically create the `labscript-suite` folder that contains initial/example labscript configurations in your home directory

```
1 labscript-profile-create
```

- to create executables for the 4 labscript applications

```
1 desktop-app install blacs runmanager runviewer lyse
```

more details:  [Regular installation \(Anaconda Cloud\) — labscript suite | the labscript suite](#)

Developer Install

- Shafin (summer student 2024) edited backend repositories to make the NI DAQ run faster (we called it “fast analog”). Refer to this page: [GitHub Sync of labscript-suite Folder](#).
- To incorporate his work, we first pull his Github remote repository to ours.
- Then, we do a developer installation of certain folders, using one of the following options with the `labscript` conda env activated and from the `labscript-suite` directory

```
1 C:\Users\wkheisenberg\labscript-suite> pip install --no-build-isolation --no-deps
2 -e labscript -e runmanager -e blacs -e lyse -e runviewer -e labscript-devices -e
labscript-utils
```

Note: if the above doesn't work, it could be because labscript, runmanager, lyse, and runviewer are already installed and not modified by Shafin's dev code. Instead, try:

```
1 pip install --no-build-isolation --no-deps -e blacs -e labscript-devices -e labscript-
utils
```

Configuring the labscript suite for first run

- Modify the `labscript-suite/labconfig/EXP-CONTROL.ini` file to change experiment name and different save directories (i.e. experiment shots, userlib, etc.) and set up the new locations you want to establish

Configuring the labscript suite for first run

- Modify the `labscript-suite/labconfig/EXP-CONTROL.ini` file to change experiment name and different save directories (i.e. experiment shots, userlib, etc.) and set up the new locations you want to establish
- Go to the connections table file (default directory specified in .ini file) and set up the connections of your hardware
 - Example connections table ([Example Connection Tables — labscript suite | labscript-devices](#))
 - Connections table overview ([Connection Table — labscript suite | labscript](#))
- After modifying the connections table, refresh runmanager and re-compile connection table in BLACs. You can then compile and execute the connections table script from runmanager into a shot file that executes in BLACs.

[Small Notes]

(1) Avoid mixing conda install and pip install

- `conda install` downloads packages **from Conda repositories** (`defaults` , `conda-forge`).
- `pip install` downloads packages **from PyPI** (`pypi.org`).
- A package from PyPI **may not be fully compatible** with one from Conda, especially for compiled libraries.

👉 **Example Problem:**

If you install `numpy` with Conda and then install `scipy` with pip, `pip` might install a version of `numpy` that conflicts with the one Conda installed.

Best Practices to Avoid Problems - Use Conda First, Then Pip Only If Necessary

1. **Use Conda First, Then Pip Only If Necessary**
2. **Never install core scientific libraries** (`numpy` , `scipy` , etc.) **via pip in a Conda environment.**

(2) Be careful not to upgrade pyzmq

- As mentioned in [this page](#), we have had troubles regarding the dependencies of pyzmq. One example problem we had was: we wanted to run “import lyse” on jupyter notebook, so we ran some codes, which upgraded pyzmq to 26.0.0, which kicked out blacs from the socket it was using. After this we ended up nuking the labscrip environment and do a fresh install.

Startup Procedure

We are running labscript in Lyman 29 (2024/08/13).

Startup Procedure

So imagine I want to start up labscript from the beginning. Currently it is launched from the terminal.

1. Open terminal
2. Open new windows for all the devices: BLACS, RunManager, and Lyse
3. Activate conda environment in each window: "conda activate labscript_dev" (Note: For the batcave the env name is "labscript")
 - a. Note: this conda environment must be installed first
4. Open up the following folders, all under \labscript-suite\: "blacs", "labscript-devices", and "labscript-utils"
 - a. Note: Shafin is using a nice terminal package called posh-git that shows you when you are in a git repository, which all three of these directories are
5. Run the labscript applications in the terminal by running "blacs", "runmanager", and "lyse"
 - a. Note: Can open up labscript applications from any terminal window with the labscript_dev environment activated
 - b. Note: there is some "error in loading MIME data", we have been safely ignoring this error so far

Compiling the Connection Table

- Go to the connections table file (default directory specified in .ini file) and set up the connections of your hardware
 - Example connections table ([Example Connection Tables — labscript suite | labscript-devices](#))
 - Connections table overview ([Connection Table — labscript suite | labscript](#))
- After modifying connections table, refresh runmanager and re-compile connection table in BLACs. You can then compile and execute the connections table script from runmanager into a shot file that executes in BLACs.

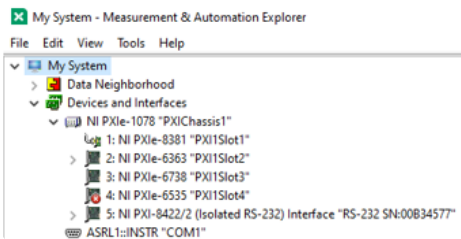
By going to [this link](#), we can find the parameters used to program NI devices in labscript.

For example, here are the parameters we can set on NI DAQ:

Class

labscript_devices.NI_DAQmx.labscript_devices.NI_DAQmx(name, parent_device=None, clock_terminal=None, MAX_name=None, static_AO=None, static_DO=None, clock_mirror_terminal=None, connected_terminals=None, acquisition_rate=None, AI_range=None, AI_range_Diff=None, AI_start_delay=0, AI_start_delay_ticks=None, AI_term='RSE', AI_term_cfg=None, AI_timebase_terminal=None, AI_timebase_rate=None, AO_range=None, max_AI_multi_chan_rate=None, max_AI_single_chan_rate=None, max_AO_sample_rate=None, max_DO_sample_rate=None, min_semiperiod_measurement=None, num_AI=0, num_AO=0, num_CI=0, ports=None, supports_buffered_AO=False, supports_buffered_DO=False, supports_semiperiod_measurement=False, supports_simultaneous_AI_sampling=False, **kwargs)

Note that MAX_name can be found by opening up NI MAX and looking in the devices neighborhood:



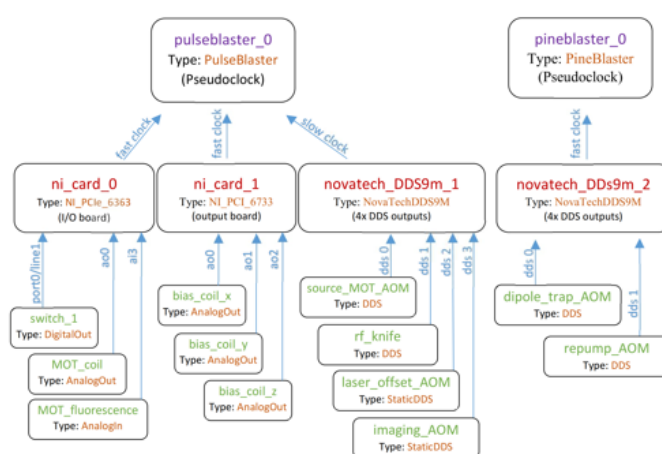
We need to add all of the analog output and digital output channels to connection_table.py. The connection for the channels should match the NI name for the channel, ie "ao1" or "ai0" or "port0/line1", while the "name" can be whatever the user wants.

How to Write the Connection Table

For more details, see the official labscript Connection Table documentation: [Connection Table — labscript suite | labscript](#)

Overview:

The **connection table** is an intuitive way to map out the way input/output devices are connected to each other in the lab. The following figure is a good visualization of a sample connection table and the connections between all the devices.



Device Hierarchy

There are 3 families of devices

PseudoclockDevice :

- These are the top most device in the hierarchy. They do not have a parent device that tells them when to update their output.
- Each **PseudoclockDevice** instance should have one or more **Pseudoclock** children. Some **PseudoclockDevice** instances may automatically create these children for you (check the device specific documentation). In turn, each **Pseudoclock** will have one or more **ClockLine** instances connected to it. These **ClockLine** instances generally refer to physical outputs of a device which will be used to clock another device. However, in some cases, one or more **ClockLine** instances may be internally created for you (check the device specific documentation).
- Any NON-pseudoclock device must have a Pseudoclock as its parent

IntermediateDevice - Devices with hardware timed outputs

- these devices are programmed to store the hardware instructions in the on-board device buffer at the start of the shot.
- the clocking signal then steps through these instructions to provide hardware timed output during the shot

Device - Devices that don't require hardware timed instructions

- they are communicated/set only during manual mode or before a shot using USB, COM, TCP/IP

Setting Up Physical Devices in Python

The connection table is defined in the `connection_table.py` file, that lives in the path defined in your labconfig `.INI` settings. The following code snippets are examples of how you can set up the basic functionalities of the devices and then define the settings to configure the device however you want.

Importing the Devices

```
1 ##### Import from the official Labscript Devices #####
2 from labscript_devices.PrawnBlaster.labscript_devices import PrawnBlaster
3 from labscript_devices.NI_DAQmx.models.NI_PXIE_6361 import NI_PXIE_6361
4 from labscript_devices.NI_DAQmx.models.NI_PCIE_6363 import NI_PCIE_6363
5 from labscript_devices.NI_DAQmx.models.NI_PXIE_6739 import NI_PXIE_6739
6 from labscript_devices.NI_DAQmx.models.NI_PXIE_6535 import NI_PXIE_6535
7
8 ##### Import from the custom RaX user_devices #####
9 from user_devices.RemoteControl.labscript_devices import RemoteControl, RemoteAnalogOut,
  RemoteAnalogMonitor
10 from user_devices.NuvuCamera.labscript_devices import NuvuCamera
```

- import the device from the labscript official devices (`labscript-devices` folder) or from user defined devices (`userlib/user_devices` folder)

Importing Other Connections/Devices:

```
1 from labscript import (
2     DigitalOut,
3     AnalogOut,
4     AnalogIn,
5     start,
6     stop,
7     add_time_marker
8 )
```

Examples

Pseudoclock

```
1 # PseudoClock
2 pb = PrawnBlaster(
3     name='pb',
```

```

4     com_port='COM5',
5     num_pseudoclocks=2
6 )
7
8 clockline_0 = pb.clocklines[0]
9 clockline_1 = pb.clocklines[1]

```

- this implicitly creates 2 **pseudoclock** devices, and a **clockline** for each.

NI Device

```

1  # NI-6361 Multi-Function Card used for.....
2  ni_6361_max_name = "PXI1Slot8" # NI card name configured in and obtained from NI MAX
   software
3  ni_6361_clockline = pb.clocklines[1]
4
5  ni_6361_mio = NI_PXIe_6361(
6      name='ni_6361_mio',
7      parent_device=ni_6361_clockline,
8      clock_terminal=f'/{ni_6361_max_name}/PFI1',
9      MAX_name=f'{ni_6361_max_name}',
10     acquisition_rate=1000e3,
11 )

```

- will implicitly set up the device based on the internal settings/capabilities of the device

Input/output Signals

```

1  ao0_6361 = AnalogOut(name='ao0_6361', parent_device=ni_6361_mio, connection='ao0')
2  ao1_6361 = AnalogOut(name='ao1_6361', parent_device=ni_6361_mio, connection='ao1')
3
4  do0_6361 = DigitalOut(name='do0_6361', parent_device=ni_6361_mio,
   connection='port0/line0')
5  do1_6361 = DigitalOut(name='do1_6361', parent_device=ni_6361_mio,
   connection='port0/line1')
6
7  ai0_6361 = AnalogIn(name='ai0_6361', parent_device=ni_6361_mio, connection='ai0')
8  ai1_6361 = AnalogIn(name='ai1_6361', parent_device=ni_6361_mio, connection='ai1')

```

Nuvu Camera

```

1  # Nuvu Camera
2  # NOTE: The initialization of the NuvuCamera creates an implicit DO under the name
   "camera_trigger" at the specified connection.
3  camera = NuvuCamera(
4      name="camera",
5      parent_device=ni_6361_mio,
6      connection="port0/line2",
7      serial_number=0xDEADBEEF, # NUVU camera initialization does not require
   serial_number, no need to touch this
8      mock=False
9  )
10 # NOTE: The NI-DAQmx driver requires an even number of digital output channels.
11 # If you have an odd number of DOs in use, create a dummy DO to satisfy this
   requirement.
12 DigitalOut(name='dummy_for_even', parent_device=ni_6361_mio, connection='port0/line3')

```

Customizable Settings

- you have a high level idea of what settings the device supports and how it can be configured.
- you have no idea how to configure those in labscript, and what the labscript drivers supports

- hovering over the device name will give you a glimpse into what parameters the device accepts.

```

##### class PrawnBlaster(
from user     name: str,
from user     trigger_device: Any | None = None,
              trigger_connection: Any | None = None,
              com_port: str = "COM1",
from labs     num_pseudoclocks: int = 1,
  Digit       out_pins: Any | None = None,
  Analo       in_pins: Any | None = None,
  Analo       clock_frequency: float = 10000000,
  start       external_clock_pin: Any | None = None,
  stop        use_wait_monitor: bool = True
  add_t      )
              PrawnBlaster Pseudoclock labscript device.
def conn     This labscript device creates Pseudoclocks based on the PrawnBlaster, a Raspberry Pi Pico with custom firmware.
# Pse       Args
pb = PrawnBlaster(
  name='pb',
  com_port='COM5',
  num_pseudoclocks=2
)

```

- you can further understand what each of the parameters does by **ctrl+click** on the device class name, which brings you to the initialization function. the doc string in the initialization function will hopefully provide you with a lot more details on the params. Here is the init and docstring of the **PrawnBlaster pseudoclock** :

```

1  def __init__(
2      self,
3      name,
4      trigger_device=None,
5      trigger_connection=None,
6      com_port="COM1",
7      num_pseudoclocks=1,
8      out_pins=None,
9      in_pins=None,
10     clock_frequency=100e6,
11     external_clock_pin=None,
12     use_wait_monitor=True,
13 ):
14     """PrawnBlaster Pseudoclock labscript device.
15
16     This labscript device creates Pseudoclocks based on the PrawnBlaster,
17     a Raspberry Pi Pico with custom firmware.
18
19     Args:
20         name (str): python variable name to assign to the PrawnBlaster
21         com_port (str): COM port assigned to the PrawnBlaster by the OS. Takes
22             the form of `COMd`, where `d` is an integer.
23         num_pseudoclocks (int): Number of pseudoclocks to create. Ranges from 1-4.
24         trigger_device (:class:`~labscript.IntermediateDevice`, optional): Device
25             that will send the hardware start trigger when using the PrawnBlaster
26             as a secondary Pseudoclock.
27         trigger_connection (str, optional): Which output of the `trigger_device`
28             is connected to the PrawnBlaster hardware trigger input.
29         out_pins (list, optional): What outpins to use for the pseudoclock outputs.
30             Must have length of at least `num_pseudoclocks`. Defaults to `[9,11,13,15]`
31         in_pins (list, optional): What inpins to use for the pseudoclock hardware
32             triggering. Must have length of at least `num_pseudoclocks`.
33             Defaults to `[0,0,0,0]`
34         clock_frequency (float, optional): Frequency of clock. Standard range
35             accepts up to 133 MHz. An experimental overclocked firmware is
36             available that allows higher frequencies.
37         external_clock_pin (int, optional): If not `None` (the default),
38             the PrawnBlaster uses an external clock on the provided pin. Valid
39             options are `20` and `22`. The external frequency must be defined
40             using `clock_frequency`.
41         use_wait_monitor (bool, optional): Configure the PrawnBlaster to
42             perform its own wait monitoring.
43

```

- it should be simple enough to read through the code associated with a param you are interested in to see how it is configured in the backend.

Convention on connection_table.py

Function conventions on connection_table.py:

- we make a function called `connection_table()` in the `connection_table.py` which contains all the devices and connections.
- this makes it easy for us to import and call `connection_table()` in any experiment script and that will correctly import over the entire connection table, allowing us access to any of the connections or definitions we defined.
- this is a good method since as the connection table changes, each of the experiment scripts will reflect those changes meaning we don't have repeat code we need to update in multiple places for each update

Also, the “if True” convention:

- we copy paste the entire connection table into each script under an if true block
- if true block allows us to leverage vscode to hide the redundant code that's copied into each file
- the good part about copy-pasting is we have access to the variable names (devices, signal connections) allowing us to use vscode code completion and function argument suggestions as we saw during our lessons
- if we import from a function, we don't have access to any of the variables and need to reference everything by name (just a string...)

Connecting hardwares to Labscript

Connecting devices using PyVISA, PySerial, and PyLablib

Note that you DO NOT have to connect and recognize all the hardware devices to labscript. For example, since labscript features precise timing, they can be used only for lasers and RF generators, while ambient temperature, humidity, radiation data can be taken with Python separately from labscript.

Labscript is **not just for communication**—it's an **experiment automation framework**. It helps physicists and engineers structure, synchronize, and automate experiments that involve multiple hardware components. If you're just talking to one device, `pyserial` or `pyvisa` is enough, but for larger experiments, Labscript **saves time and reduces complexity**.

Would you like help setting up Labscript for a specific experiment? 😊

📄 🍌 🗨️ 🔊 🔗 🔄

can you add data taken by pyserial and others into a h5 file created in labscript?

Yes! You can **add data collected via `pyserial`, `pyvisa`, or other Python libraries into an HDF5 (.h5) file created by Labscript**. Labscript already uses HDF5 to store experimental parameters and results, and you can extend this file to include external data.

- Good tutorial [video](#) and its [Github](#) page about PyVISA/PySerial
- Good tutorial [video](#) for connecting to NI DAQ
- Good tutorial notes for connecting to NI PCIe card in the [Drive folder](#)
- **This [PyLablib folder](#) contains the codes/instructions for commonly used gauges, lasers, oscilloscopes, cameras, etc.**

Differences between Pyserial, PyVISA, and PyLablib

Basically, Pyserial is for serial connection (RS232), PyVISA is for USB and Ethernet, and PyLablib is a new, combined package but for a smaller application range (specific companies/devices).

Below is what ChatGPT says. Don't believe it is 100% correct, but generically gives an overall comparison:

◆ Which One Should You Use?

Feature	pyserial	pyvisa	pylablib
Simple Serial (RS-232, USB-serial)	✔ Yes	⚠ Limited	✔ Yes
GPIB support	✗ No	✔ Yes	✔ Yes
USB-TMC support	✗ No	✔ Yes	✔ Yes
Ethernet support	✗ No	✔ Yes	✔ Yes
Built-in drivers for instruments	✗ No	✗ No	✔ Yes
SCPI command support	⚠ Manual	✔ Yes	✔ Yes
Works with Arduino & Microcontrollers	✔ Yes	✗ No	✗ No

📌 TL;DR – When to Use What?

- Use `pyserial` if your device only speaks **basic serial** (RS-232, USB-serial, UART).
- Use `pyvisa` if your device is a **lab instrument** with SCPI commands over GPIB, USB-TMC, or Ethernet.
- Use `pylablib` if your device is a **supported scientific instrument** (motion stages, laser controllers, etc.).

External Software Devices

- 1 [Introduction](#)
- 2 [How is the communication done?](#)
 - 2.1 [Synchronous Communication](#)
 - 2.2 [Asynchronous Communication](#)
- 3 [Features that can be implemented using the communication protocols:](#)
- 4 [Feature Implementation Examples](#)
 - 4.1 [Manual Control of LabVIEW Laser Lock:](#)
 - 4.2 [Reflecting UNEXPECTED changes from ExternalSoftware GUI to BLACS GUI](#)
 - 4.3 [Reflecting EXPECTED changes from ExternalSoftware GUI to BLACS GUI](#)
 - 4.4 [Setting LabVIEW Laser Lock Before a Shot:](#)
 - 4.5 [Setting Ablation Laser Coords before Shot #1:](#)
 - 4.6 [Setting Ablation Laser Coords Before Shot #2:](#)
 - 4.7 [Keeping track of values for each shot:](#)

Introduction

The `ExternalSoftware` (formerly known as `RemoteControl`) `user_devices` provides a way to perform synchronous and asynchronous communication between `labscript BLACS` and a program/software/GUI that is *not* implemented in `labscript`.

How is the communication done?

The **client** is the `BLACS` software, and they communicate with the **server** which is hosted on the external software.

The **server** opens up TCP/IP connections on a specific port, and the **client** binds to those ports.

Synchronous Communication

Implemented using `REQ-REP` sockets.

- The **BLACS client** sends requests to the **external software server**. **BLACS client** waits for a response, this call is blocking
- The **external software server** processes those requests, then tells the BLACS client the request has been processed.
- The **BLACS client** hears back a response and can continue with program execution
- Barring any processing time, the `REQ-REP` protocol typically completes within milliseconds. It's **used for fast, low-latency, “real-time” communication**

Asynchronous Communication

Implemented using `PUB-SUB` sockets.

- The **external software publisher** publishes data to certain topics
- The **BLACS client** can have multiple subscribers for as many topics they want to subscribe to
- When data is published, the **BLACS client subscribers** receive those values and do whatever with them
- There is typically a 300-500ms delay between when data is first published and when the subscriber reads it. **Perfectly fine for human monitoring.**

Features that can be implemented using the communication protocols:

Recall, `BLACS` allows for manual and buffered (hardware-timed) control of devices. The `ExternalSoftware` allows for both of these.

It is also crucial that our main Experiment Control/Data Acquisition Software has information of all the devices at all times, especially when saving shot data.

- The `ExternalSoftware` ensures that we query and then save the current state of each `ExternalSoftware` when in their `transition_to_buffered`. If necessary, we can easily add a query to the state of the `ExternalSoftware` at the end of a shot. It should be simple to add in state machine flows to react to changes to the state during the shot.

 Each external software is unique, and the way we implement features such as manual/buffered mode programming or device querying will be different.

Feature Implementation Examples

Let's go through a few examples to understand how communication protocols are used to achieve the aforementioned features.

Manual Control of LabVIEW Laser Lock:

- You make a change to the laser frequency in the `BLACS tab`
- This calls the `program_manual BLACS worker` function.
- Inside `program_manual` we send a **REQUEST** to LabVIEW to change the laser frequency to the new value.

- LabVIEW receives the request and internally changes the laser frequency to the request value.
- LabVIEW **RESPONDS** that the new laser frequency has been set
- BLACS receives confirmation and moves

 A very similar structure can be used for manual control of Laser Raster program coordinates

Reflecting UNEXPECTED changes from ExternalSoftware GUI to BLACS GUI

- you go into LabVIEW or Laser Raster program GUI and directly modify the laser frequency/coordinates instead of using BLACS.
- For the setpoints connections, we send a **REQUEST** to poll the values of the ExternalSoftware. This is done at a low frequency (0.2Hz). The ExternalSoftware **RESPONDS** with the value
- If BLACS and ExternalSoftware GUI are mismatched we warn the user in BLACS and the user can resolve it accordingly.

Reflecting EXPECTED changes from ExternalSoftware GUI to BLACS GUI

- the actual frequency of lasers are constantly changing in LabVIEW.
- **each** change results in the LabVIEW program to **PUBLISH** the new value to the corresponding topic named under the laser connection table name
- the BLACS tab **SUBSCRIBER** receives a value and updates the GUI tab accordingly.

 As you can see, the asynchronous PUB-SUB communication is only used for manual mode feedback.

Setting LabVIEW Laser Lock Before a Shot:

- The user defines in the `labscript` sequence definition script to set the laser to a certain value (`laser.set_frequency(value)`). This can be a constant value (`390.01238 nm`), or can be a `runmanager` variable (`LASER_FREQ_SWEEP`).
- The shot is executed and the state machine starts, calling the `transition_to_buffered` `BLACS worker` function.
- In `transition_to_buffered` the .h5 file is unpacked for the value that each laser needs to be set to. A **REQUEST** is sent to set the value
- Once a **RESPONSE** is received, we consider the Laser Lock device to be done transitioning to buffered.

Setting Ablation Laser Coords before Shot #1:

- The user defines in the `labscript` sequence definition script to set the laser position to a certain coordinate (`laser.set_x_coord(value)` , `laser.set_y_coord(value)`).
 - This can be a constant value (`2.120mm`), or can be a `runmanager` variable (`LASER_X_COORD`).
- The shot is executed and the state machine starts, calling the `transition_to_buffered` `BLACS worker` function.
- In `transition_to_buffered` the .h5 file is unpacked for the value that each coordinate needs to be set to. A **REQUEST** is sent to set each value.

i It might make more sense to send both coordinates at the same time in one request. You would need to modify the Laser Raster program server side handling to accept the two values and then move in one go, rather than moving to x and y separately.

- The Laser Raster program receives the new coordinates. It makes the driver calls to move the motors. Once the motors have stopped moving, send a **RESPONSE** saying its complete.
- Once a **RESPONSE** is received, we consider the Laser Raster device to be done transitioning to buffered.

Setting Ablation Laser Coords Before Shot #2:

- The user defines in the `labscript` sequence definition script, and they want the ablation laser to move, but not to a particular coordinate. The coordinate will be handled on the Laser Raster program side. The use can specify `laser.trigger_laser_movement(bool)` in the sequence.
 - This can be a constant Boolean value or can be a `runmanager` variable (`LASER_MOVEMENT=[true, true, false, false]`).
- The shot is executed and the state machine starts, calling the `transition_to_buffered` `BLACS worker` function.
- In `transition_to_buffered` the .h5 file is unpacked for the value that each coordinate needs to be set to. We see that we need to tell the Laser Raster program it needs to move to its next spot. A **REQUEST** is sent to communicate this.
- The Laser Raster program receives command. It internally queries what the next position is and makes the driver calls to move the motors. Once the motors have stopped moving, send a **RESPONSE** saying it has been moved to `[NEW_X, NEW_Y]` .

- Once a **RESPONSE** is received we save the values in an internal variable. we consider the Laser Raster device to be done transitioning to buffered.
- When the shot is complete, we save the internal variable value containing the new coordinates in the .h5 file.

Keeping track of values for each shot:

- the external software is connected, but not being used in the shot. This can occur when you are not experimenting with laser frequency, you just have a value set inside LabVIEW. You are not setting it each shot.
- When the **BLACS state machine** begins executing the shot, it will call **transition_to_buffered** for ALL devices in the sequence connection table.
- In the **ExternalSoftware transition_to_buffered** we analyze the .h5 file and determine there is nothing for us to set.
- Instead of sending a **REQUEST** of the value to set the **ExternalSoftware** to, we just query ALL the values and save them in the .h5 file at the end of a shot, in the **post_experiment BLACS worker** function.

NuVu camera notes

- Use SDK to write python functions with same name as DLL functions
- cameras cannot be initialized until the internal temp has reached a certain value

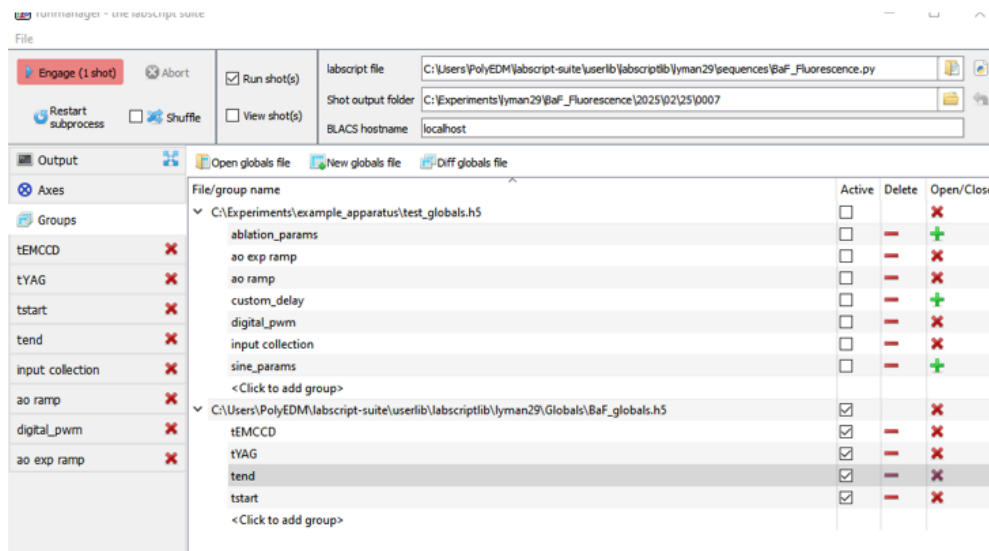
Communicating with LabView

[↗ send data from LabView to Python and get back](#)

How to set measurement parameters

I think it is more convenient for us to set measurement parameters as a global variable rather than defining in the sequences file for the following two reasons:

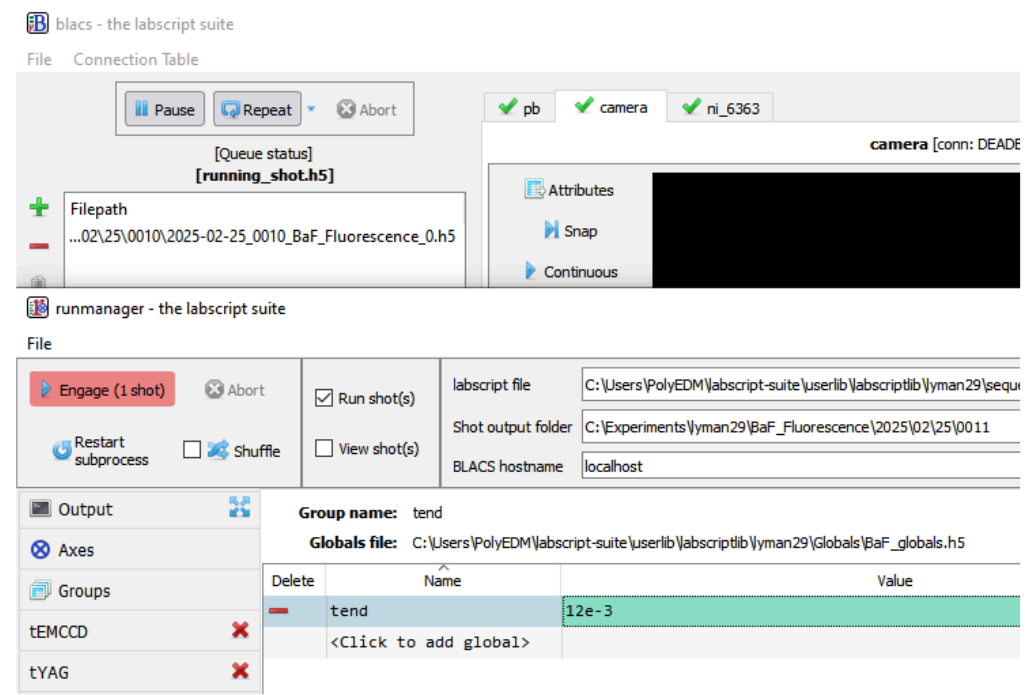
- (1) We can set global variables in runmanager screen for each shot, so this way we can keep track of the parameters without opening VSCode
- (2) Global variables are automatically saved in the h5 file, so we don't have to work hard in writing lines to save variables from the sequence file



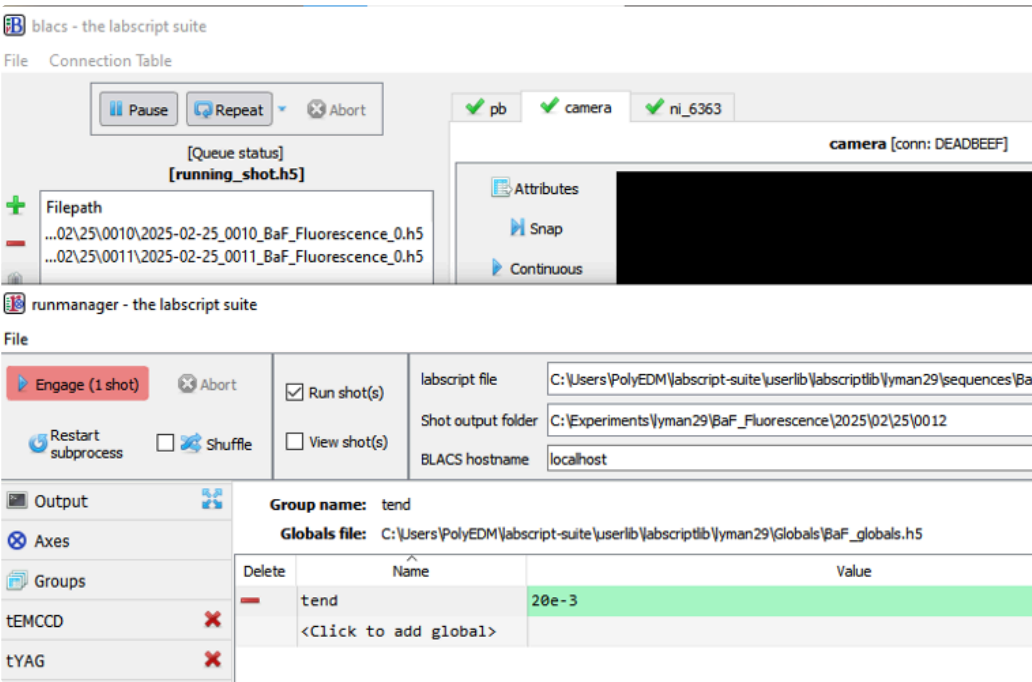
Global variables are shown in "Groups". You can add a global variable by clicking <Click to add group>. You can also edit values by clicking the existing global variables. After you edit these and close Runmanager you will get a pop-up asking if it can rewrite the .ini file - say Yes.

Below I will demonstrate (1). Let us say we want to measure in-cell absorption signal for durations $tend - tstart = 12e-3$ sec and $20e-3$ sec, and I want to take these data at the same time.

To do this, I set $(tstart, tend) = (0, 12e-3)$ for ...0010_BaF_Fluorescence_0.h5 shot and $(0, 20e-3)$ for ...0011_BaF_Fluorescence.h5 shot.



Setting tend=12e-3 for ...0010_BaF_Fluorescence.h5 shot



Setting tend=20e-3 for ...0011_BaF_Fluorescence.h5 shot

Then I run blacs, and I get a .h5 file that measured with tend=12e-3 s for ..0010.. shot and tend=20e-3 s for ..0011.. shot.

Open HDF5

Help

Opened files

2025-02-25_0010_BaF_Fl...

connection table

data

- ni_6363
- traces

BaF X-B Absorption

devices

- camera
- ni_6363
- pb
- front_panel
- globals
- images
- labscriptlib

script

- shot_properties

time_markers

waits

x D0

1181	1.181e-2	1.929e-2
1182	1.182e-2	1.897e-2
1183	1.183e-2	1.897e-2
1184	1.184e-2	1.994e-2
1185	1.185e-2	1.961e-2
1186	1.186e-2	1.897e-2
1187	1.187e-2	1.897e-2
1188	1.188e-2	1.994e-2
1189	1.189e-2	1.961e-2
1190	1.190e-2	2.026e-2
1191	1.191e-2	1.994e-2
1192	1.192e-2	1.897e-2
1193	1.193e-2	2.058e-2
1194	1.194e-2	1.994e-2
1195	1.195e-2	1.929e-2
1196	1.196e-2	1.994e-2
1197	1.197e-2	1.929e-2
1198	1.198e-2	2.026e-2
1199	1.199e-2	2.058e-2

Made by PaNOSC at ESRF

Opened files

2025-02-25_0011_...

2025-02-25_0011_BaF_Flu...

2025-02-25_0011_BaF_Flu...

2025-02-25_0011_BaF_Flu...

traces

BaF X-B Absorption

devices

- front_panel

globals

- images

labscriptlib

script

- shot_properties

time_markers

waits

1984	1.984e-2	1.929e-2
1985	1.985e-2	1.929e-2
1986	1.986e-2	1.961e-2
1987	1.987e-2	1.897e-2
1988	1.988e-2	1.768e-2
1989	1.989e-2	1.929e-2
1990	1.990e-2	1.736e-2
1991	1.991e-2	1.704e-2
1992	1.992e-2	1.671e-2
1993	1.993e-2	1.607e-2
1994	1.994e-2	1.542e-2
1995	1.995e-2	1.639e-2
1996	1.996e-2	1.768e-2
1997	1.997e-2	1.897e-2
1998	1.998e-2	1.832e-2
1999	1.999e-2	1.736e-2

Made by PaNOSC at ESRF

Template

[Date] MM/DD/YYYY

[Person] XX (write down all people who were there during the debugging)

[Computer] {Lyman 29 / Batcave / Oxford 38 / Oxford 60}

[Error Type] {Labview installation / Blacs / Lyse / Hardware / Other / Unknown}

[Summary] (In one or two sentences, describe the problem, whether it is one-time or recurring, and the solution.)

Details

(Example below)

We were looking for BaF X-B fluorescence using labscript for the first time. OneNote page is 2025-02-07 BaF Fluorescence with Labscript ([Web view](#))

In sequence BaF_Fluorescence.py, we are running into errors with camera acquisition.

Exception in worker - Fri Feb 07, 20:17:33

```

1  Traceback (most recent call last):
2    File "C:\Users\PolyEDM\labscript-suite\userlib\user_devices\NuvuCamera\blacs_workers.py", line 245, in
    transition_to_manual
3      self.set_attributes_smart(self.manual_mode_camera_attributes)
4    File "C:\Users\PolyEDM\conda\envs\labscript_dev\Lib\site-packages\labscript_devices\IMAQdxCamera\blacs_workers.py", line 302, in
    set_attributes_smart
5      self.camera.set_attributes(uncached_attributes)
6    File "C:\Users\PolyEDM\labscript-suite\userlib\user_devices\NuvuCamera\blacs_workers.py", line 35, in set_attributes
7      self.camera_utils.set_attributes(attr_dict)
8    File "C:\Users\PolyEDM\labscript-suite\userlib\user_devices\NuvuCamera\Nuvu_sdk\Nuvu_cam_utils.py", line 57, in
    set_attributes
9      self.attribute_setters[attr](value)
10   File "C:\Users\PolyEDM\labscript-suite\userlib\user_devices\NuvuCamera\Nuvu_sdk\Nuvu_cam_utils.py", line 71, in func
11     raise e from None
12   File "C:\Users\PolyEDM\labscript-suite\userlib\user_devices\NuvuCamera\Nuvu_sdk\Nuvu_cam_utils.py", line 67, in func
13     return f(*args, **kwargs)
14         ^^^^^^^^^^^^^^^^^^^^^
15   File "C:\Users\PolyEDM\labscript-suite\userlib\user_devices\NuvuCamera\Nuvu_sdk\Nuvu_cam_utils.py", line 161, in
    set_exposure_time
16     self.setExposureTime(new_exposure_time)
17   File "C:\Users\PolyEDM\labscript-suite\userlib\user_devices\NuvuCamera\Nuvu_sdk\nc_camera.py", line 232, in
    setExposureTime
18     self.errorHandling(nuvuException.value())

```

Page 48 of 4

```

1 Traceback (most recent call last):
2   File "C:\Users\PolyEDM\.conda\envs\labscript_dev\Lib\threading.py", line 982, in run
3     self._target(*self._args, **self._kwargs)
4   File "C:\Users\PolyEDM\labscript-
suite\userlib\user_devices\NuvuCamera\blacs_workers.py", line 81, in grab_multiple
5     images.append(self.grab())
6         ^^^^^^^^^^^^^^^
7   File "C:\Users\PolyEDM\labscript-
suite\userlib\user_devices\NuvuCamera\blacs_workers.py", line 66, in grab
8     return self.camera_utils.get_queued_image()
9         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
10  File "C:\Users\PolyEDM\labscript-
suite\userlib\user_devices\NuvuCamera\Nuvu_sdk\Nuvu_cam_utils.py", line 71, in func
11    raise e from None
12  File "C:\Users\PolyEDM\labscript-
suite\userlib\user_devices\NuvuCamera\Nuvu_sdk\Nuvu_cam_utils.py", line 67, in func
13    return f(*args, **kwargs)
14        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
15  File "C:\Users\PolyEDM\labscript-
suite\userlib\user_devices\NuvuCamera\Nuvu_sdk\Nuvu_cam_utils.py", line 199, in
get_queued_image
16    return self.getImg()
17        ^^^^^^^^^^^^^^^
18  File "C:\Users\PolyEDM\labscript-
suite\userlib\user_devices\NuvuCamera\Nuvu_sdk\nc_camera.py", line 397, in getImg
19    self.read()
20  File "C:\Users\PolyEDM\labscript-
suite\userlib\user_devices\NuvuCamera\Nuvu_sdk\nc_camera.py", line 390, in read
21    self.errorHandling(nuvuException.value())
22  File "C:\Users\PolyEDM\labscript-
suite\userlib\user_devices\NuvuCamera\Nuvu_sdk\nc_camera.py", line 113, in errorHandling

```

```

23     raise NuvuException("Error Code: " + str(error)+ ". \n Refer to error.h file in Nuvu
    SDK documentation.")
24 user_devices.NuvuCamera.Nuvu_sdk.nc_camera.NuvuException: 'Error Code: 214. \n Refer to
    error.h file in Nuvu SDK documentation.'
```

02/10/2025

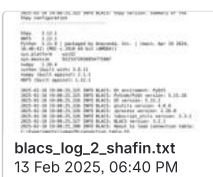
I talked with Shafin, he gave the following steps for debugging:

Camera Triggered Mode Issues

Here is the relevant part of the connection table we are using:

```

1  # Nuvu Camera
2  # NOTE: The initialization of the NuvuCamera creates an implicit DO under the name
   "camera_trigger" at the specified connection.
3  camera = NuvuCamera(
4      name="camera",
5      parent_device=ni_6363,
6      connection="port0/line2",
7      serial_number=0xDEADBEEF, # NUVU camera initialization does not require
   serial_number, no need to touch this
8      camera_attributes={
9          "readoutMode":1, #1 = EM
10         "exposure_time":20, #Shafin: "Um miliseconds?"
11         "timeout": 1000, #See above for units
12         "square_bin": 1, #NxN bin size
13         "target_detector_temp":-60,
14         "emccd_gain": 500, #Max 5000
15         "trigger_mode":1, # 1 = EXT_LOW_HIGH, #0 = INT, 2 "EXT_LOW_HIGH_EXP" (minus
   for HIGH_LOW)
16     },
17     manual_mode_camera_attributes={
18         "readoutMode":1,
19         "exposure_time":20,
20         "timeout": 1000,
21         "square_bin": 1,
22         "target_detector_temp":-60,
23         "emccd_gain": 500,
24         "trigger_mode":0,
25     },
26     mock=False
27 )
```



Basic idea of issues:

1. There was a flag that wasn't being updated properly in the configure acquisition step
2. Then we ran into more issues that led us to figure out that we did not install Shafin's labscript code, just the regular labscript code. Hence no fast analog input functionality
3. As a result, because Shafin added a post-experiment step that saves the camera image, we are just going to manual instead which does not have image saving, so we do not save

the image

We installed Shafin's code by opening up labscript-suite, activating conda environment, and running

```
pip install --no-build-isolation --no-deps -e blacs -e labscript-devices -e labscript-utils
```

And then once we recompiled everything, including run manager, it worked.

Package dependence issue for installing labscript

[Date] 07/16/2024

[Person] Shafin

[Computer] Lyman 29

[Error Type] Labview installation

[Summary] There was a package version incompatibility between dependencies installed via conda install and pip install. Solved by selecting specific version and partially installing via pip install.

Details

Conda yml files:

- conda install of labscript-suite internally installs the h5py package from conda. The conda install of h5py was bad, so launching labscript application were failing on h5py import due to dll incompatibilities.
 - This is a known issue with h5py conda installs: `ImportError: DLL load failed while importing defs: The specified procedure could not be found`

Regular Python pip installs:

- The labscript-suite pip install which internally installs the pyzmq dependency defaults to installing the 26.3.0 version. Labscript applications were failing on launch due to issues with the internal server connections between applications
- Was not able to downgrade the pip version of pyzmq to the functioning 23.2.0 because of a weird Visual Studio C++ Build Tool incompatibility (out of date).
 - Trying to follow the instructions to install a newer build tool cited an Operating System incompatibility. Idk why considering the install was for the correct cpu architecture...

The solution was to only conda install a subset of the packages in conda and the rest using pip in the conda environment.

Initial conda environment.yml:

- Pyzmq=23.2.0
- Python=3.11.9

Pip install:

- Labscript-suite
 - Also handles the h5py dependency from the right source

To get the developer install, follow the regular python instructions (Python Package Index) in the conda environment since we are using pip install for labscript-suite

Everything else is working exactly as expected

Packages listed in the Labscript conda environment:

 conda_list_output.txt

Debugging Issues

We installed Shafin's code by opening up labsript-suite, activating conda environment, and running `pip install --no-build-isolation --no-deps -e blacs -e labsript-devices -e labsript-utils`

And then once we recompiled everything, including run manager, then it worked!

→ Was there a problem in the installation procedure? Is it up-to-date?

Intro to NI DAQ

Read [this](#) for introduction in DAQ

Also [this](#) is good intro to terminology

AI/DI: Analog/Digital input

AO/DO: Analog/Digital output

PFI: Programmable function interface

PO (port 0): basically a digital output/input, where output and input can be specified by the DAQ.

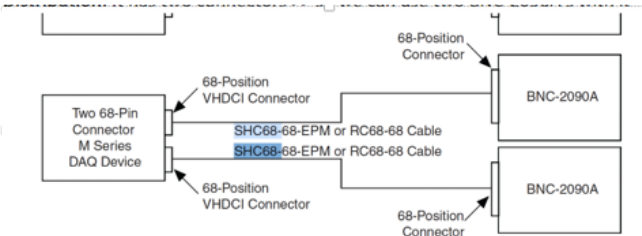
NI DAQ chassis in 38 Oxford is connected to the desktop via fiber connection [NI PXIe-8398](#)
(Part number: 784179-01 and 784179-01)

Analog output

The chassis has [NI PXIe-6738](#) connected to NI BNC-2090A (BNC terminal block: basically a lot of BNC and screw connections where you can receive and send signals with programmed functions like high pass filter if used in combination with the DAQ chassis) via cable PXIe-6361.

PXIe-6738: 32 channel 16bit, 1MS/s for 8 channel and 350kS/s for 32 channels. Channels are grouped into banks of 4 and therefore 1 out of 4 can sustain max rate, or it will get slower

The chassis also has [NI PXIe-6739](#) which is essentially two 6738 cards in one. This is currently connected to NI SCB-68A which is a terminal block like BNC-2090A but only has screw connections



Sample connection to NI PXIe-6739

PXIe-6739: 64 channel 16bit, 1MS/s for 16 channel and 350kS/s for 64 channels. Channels are grouped into banks of 4 and therefore 1 out of 4 can sustain max rate, or it will get slower

Terminal block acronyms:

P0.<0..31>	D GND	Input or Output	Port 0 Digital I/O Channels —You can individually configure each signal as an input or output. Refer to the Connecting Digital I/O Signals section of Chapter 6, <i>Digital I/O</i> .
APFI <0,1>	AO GND or AI GND	Input	Analog Programmable Function Interface Channels —Each APFI signal can be used as AO external reference inputs for AO <0..3>, AO external offset input, or as an analog trigger input. APFI <0,1> are referenced to AI GND when they are used as analog trigger inputs. APFI <0,1> are referenced to AO GND when they are used as AO external offset or reference inputs. These functions are not available on all devices; refer to the specifications for your device. Refer to the APFI <0,1> Terminals section of Chapter 11, <i>Triggering</i> .

PFI <0..7>/ P1.<0..7>	D GND	Input or Output	Programmable Function Interface or Port 1 Digital I/O Channels —Each of these terminals can be individually configured as a PFI terminal or a digital I/O terminal. As an input, each PFI terminal can be used to supply an external source for AI, AO, DI, and DO timing signals or counter/timer inputs. As a PFI output, you can route many different internal AI, AO, DI, or DO timing signals to each PFI terminal. You also can route the counter/timer outputs to each PFI terminal. As a Port 1 digital I/O signal, you can individually configure each signal as an input or output. Refer to the Connecting Digital I/O Signals section of Chapter 6, <i>Digital I/O</i> , or to Chapter 8, <i>PFI</i> .
--------------------------	-------	-----------------	---

A buffer is a temporary storage in computer memory for acquired or to-be-generated samples. Typically this storage is allocated from your computer's memory and is also called the task buffer.

Digital output

[NI PXIe-6535](#): 32 DIO, 10MS/s, 40MB/s total bandwidth

Note that this hits 3.3V instead of the multifunction card 2.2 V

Maybe will make acquisition simpler as this card is only OUT, the other card is fully IN

AIO, multifunction

[NI PXIe-6363](#): Also what YbOH already uses. 32AI / 48 DIO, 4AO

AI: 2MS/s max, 1MS/s aggregate

>10 GΩ in parallel with 100 pF